

When I ran this code on my consumer-grade laptop, it ran in about 0.0017 seconds. This is a reasonable way to express how good insertion sort is—it can fully sort a list with eight items in 0.0017 seconds. If we want to compare insertion sort with any other sorting algorithm, we can compare the results of this `timeit` timing to see which is faster, and say the faster one is better.

However, using these timings to compare algorithm performance has some problems. For example, when I ran the timing code a second time on my laptop, I found that it ran in 0.0008 seconds. A third time, I found that it ran on another computer in 0.03 seconds. The precise timing you get depends on the speed and architecture of your hardware, the current load on your operating system (OS), the version of Python you're running, the OS's internal task schedulers, the efficiency of your code, and probably other chaotic vagaries of randomness and electron motion and the phases of the moon. Since we can get very different results in each timing attempt, it's hard to rely on timings to communicate about algorithms' comparative efficiency. One programmer may brag that they can sort a list in *Y* seconds, while another programmer laughs and says that their algorithm gets better performance in *Z* seconds. We might find out they are running exactly the same code, but on different hardware at different times, so their comparison is not of algorithm efficiency but rather of hardware speed and luck.

Counting Steps

Instead of using timings in seconds, a more reliable measure of algorithm performance is the number of steps required to execute the algorithm. The number of steps an algorithm takes is a feature of the algorithm itself and isn't dependent on the hardware architecture or even necessarily on the programming language. [Listing 4-3](#) is our insertion sort code from Listings 4-1 and 4-2 with several lines added where we have specified `stepcounter+=1`. We increase our step counter every time we pick up a new file to insert from the old cabinet, every time we compare that file to another file in the new cabinet, and every time we insert the file into the new cabinet.

```
def insert_cabinet(cabinet,to_insert):
    check_location = len(cabinet) - 1
    insert_location = 0
    global stepcounter
```